netsentinel ��� Project Report

Project Report — Computer Networks Course Project

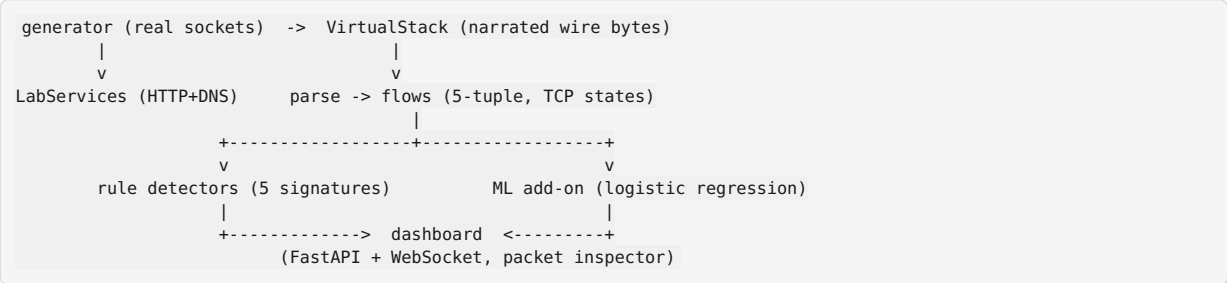
Mahesh Thakur — IIT Patna, B.Tech MnC (2024–2028)

October 2026

Abstract

netsentinel is a self-contained network security laboratory: every packet is constructed and parsed at the byte level (IPv4, TCP, UDP, ICMP, DNS), all traffic moves through real localhost sockets, flows are assembled by 5-tuple, and six traffic scenarios — normal browsing plus five attacks — are caught by rule-based detectors and a from-scratch logistic-regression add-on, side by side, on a live dashboard. A reproducible experiment suite measures detection thresholds, learning curves, and feature value; every number in this report comes from the logged run `20261020-025510` (git `a13e818`).

Architecture



- `netsentinel/packets.py` — byte-level builders/parsers for IPv4 (RFC 791), TCP (RFC 793), UDP (RFC 768), ICMP (RFC 792), DNS (RFC 1035). Header checksums per RFC 1071; DNS label encoding; every parser round-trips in tests.
- `netsentinel/stack.py` — a virtual TCP stack: since unprivileged processes cannot sniff loopback, it emits exactly the segments the kernel would exchange, with correct sequence/acknowledgement arithmetic (SYN and FIN each consume one sequence number).
- `netsentinel/flows.py` — bidirectional flow assembly on canonicalized 5-tuples plus a TCP state machine; half-open connections are the SYN-flood fingerprint.
- `netsentinel/generator.py` — real sockets: HTTP fetches, DNS queries, `connect()` port scans, real UDP blasts; crafted-packet attacks (SYN flood, ping sweep) inject the identical wire bytes a raw socket would send.
- `netsentinel/detectors.py` — five behavioural rules plus the ML add-on.
- `netsentinel/experiments.py` — the logged, reproducible evaluation harness.

Scenarios and detectors

Scenario	What really happens	Detection signal
Normal browsing	real HTTP sessions + DNS lookups	— (control)
Port scan	real `connect()` across ports 1–100	SYN fan-out: distinct ports per source
SYN flood	150 crafted SYNs, spoofed TEST-NET sources	half-open count, SYNs per target
DNS tunnel	secret base32-encoded into query labels	label length + Shannon entropy
Ping sweep	ICMP echo fan-out across 10.0.2.0/24	distinct echo destinations per source
UDP flood	200 datagrams into one service port	datagram rate into a single port

The ML add-on

Logistic regression implemented from scratch (`p = sigmoid(w·x + b)` , cross-entropy loss, gradient descent) over 13 window features: protocol counts, SYN/UDP/ICMP fan-out maxima, DNS label length and entropy. Features are z-scored with training-set statistics. It exists to be compared with the rules, not to replace them.

Experiments

All results below were produced by `python run_experiments.py` and logged to `docs/experiments/results.json` (run `20261020-025510` , 72 labelled windows, 12 per class, 23.2s wall time).

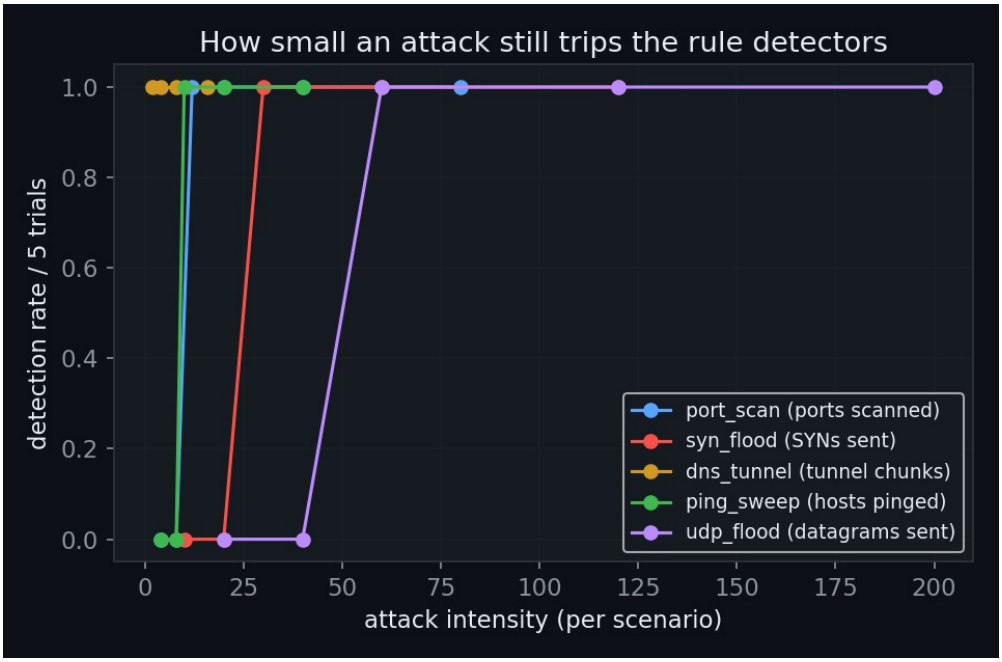
Rules vs ML, head-to-head

On held-out windows both approaches separate the lab’s traffic (precision, recall, F1 = 100.0% for the rules and 100.0% for the ML add-on; AUC 1.0). Stratified 4-fold cross-validation gives F1 100.0% ± 0.0. **This measures separability of this lab’s traffic, not**

of the open internet — the generator's distribution is the only world the model has seen.

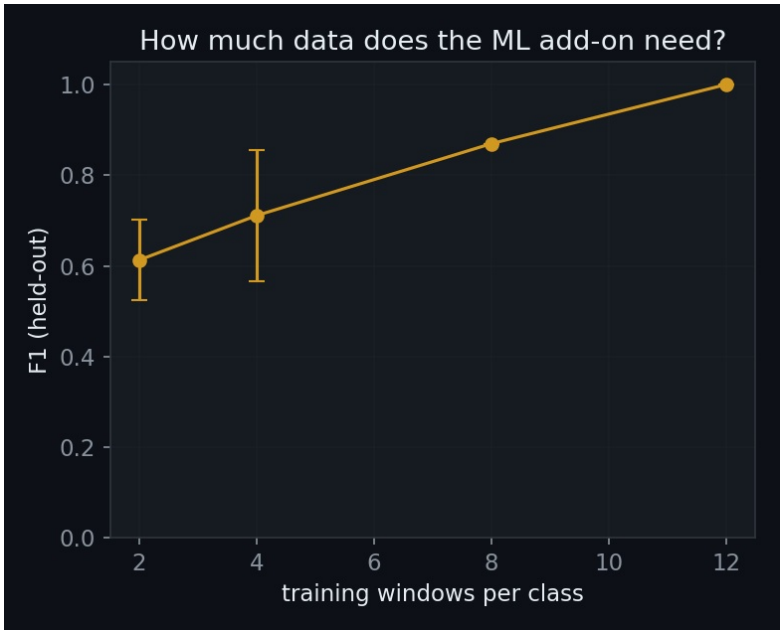
Detection vs attack intensity

The most honest experiment: how small can an attack be and still trip the rule? Each point is 5 fresh trials through the live pipeline.



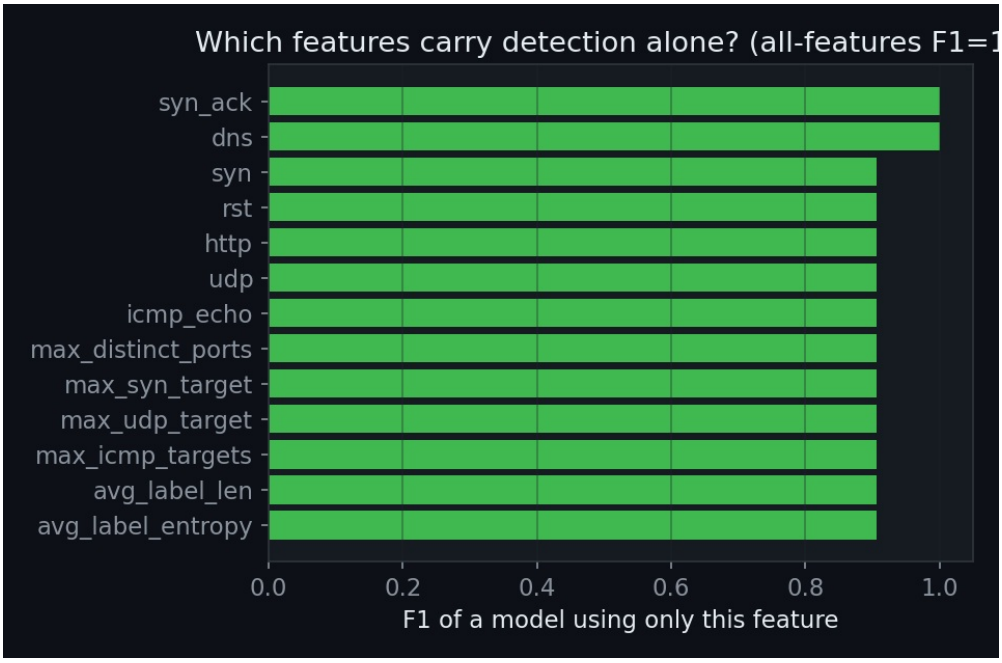
Detection rate vs attack intensity

Data scaling



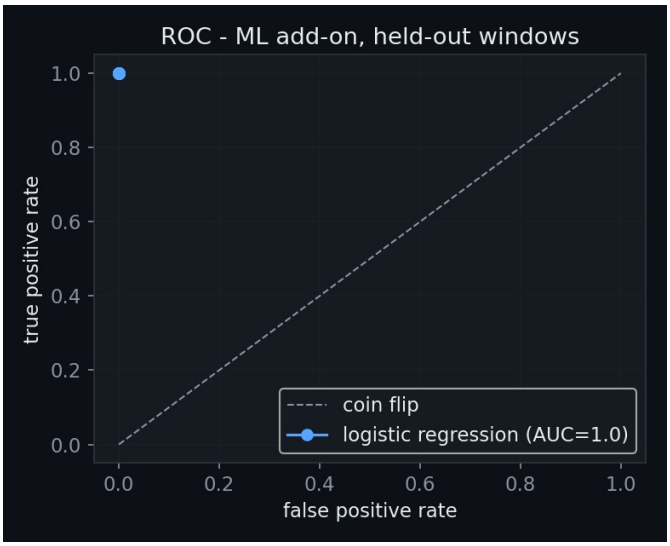
Held-out F1 vs training windows per class

Which features carry detection alone



F1 of single-feature models

ROC



ROC on held-out windows

Honest limitations

- Loopback cannot be sniffed without root; the VirtualStack narrates the exact wire bytes instead. The analysis pipeline is identical to a pcap-fed one.
- ML scores reflect the lab distribution. Concept drift on real traffic would degrade them; the report shows learning curves and thresholds rather than one vanity number.
- UDP/53 traffic that is not parseable DNS is labelled `UDP/53` and counted as UDP — itself a mild anomaly signal.

Reproducing everything

```
pip install -r requirements.txt
python train_window_model.py       # retrain the ML add-on
python run_experiments.py          # regenerate every number and chart
python -m pytest tests -q          # 27 tests
uvicorn server:app --port 8000     # live dashboard
docker build -t netsentinel . && docker run -p 8000:8000 netsentinel
```